INT161 · Basic Backend Development

Mock Exam Simulation

Follow the lecturer's practice exam in order: get the files, load the database, read the spec, run the checker, and do question 1 live.

What You Get | Load the Database | Read the Spec | Run the Checker | Question 1 Live

From Sample to Exam | Rules and Rehearsal | Glossary

01

What You Get

The lecturer ran the practice like a fire drill. Relaxed now, so the real day has no surprises. You download four things.

`Your spec`

A .md file with your own ID, name and secret key. Open it in preview mode.

`init-database.sql`

One file for everyone. It creates the database and fills it.

`The template`

A project with layers already set up. Yours is named after you.

`tests/ folder`

The **checker**. Copy the whole folder into your project.

- The spec is personal. Each student has an ID and a secret key in it. The lecturer compares them to his own table when he grades.

- He opened it in VS Code preview. Use preview, so the tables and code blocks are easy to read.
- The sample system was a mobile phone shop: brands and products. On exam day the topic may change, but expect two tables and one one-to-many link.

> ⚠ **Your name is not here**
>
> This page uses a made-up person: Somchai Jaidee, ID 67000000000, key AB12cd. Swap in the values from your own spec. Never use another student's.

02

Load the Database

First job: load the database. If this fails you cannot do the exam.

The spec says to run this command.

from the spec

```
$ mysql -u root -p < init-database.sql
```

The lecturer tried it on his machine, live. It failed.

1. The terminal did not know mysql. The MySQL bin folder was not in **PATH**.
2. He found MySQL in Program Files and added its bin folder to his user PATH. He opened a new terminal.
3. The command was found now, but the login still failed. He dropped this way.
4. He used the IDE instead. Right click the data source, Run SQL Script, pick the file, then refresh.

- After the refresh you should see mobile_distributor with two tables, brands and products. His brands table had 22 rows.
- At the exam the **connection** may not exist yet. Create the data source before the day.
- His point: you learned SQL in this course. Running a script is all it takes. Any tool that can run the file is fine.

> **On VS Code and Windows**
>
> The lecturer used WebStorm. VS Code has no data source. Use MySQL Workbench, or cmd /c, or source. The Exam Walkthrough, section 2, has each step.

03

Read the Spec

Read the whole spec before you write code. Whatever the spec says, you do.

- Put your ID in a variable in app.js.
- Every **route** starts with /api/your-id. This is the base path. The rest is the **resource**.
- Wrong base path means the checker cannot reach you. Every test fails.
- Build the one **endpoint** that returns your info: GET /api/your-id/student-info. Status 200. It is worth 10 points.
- The lecturer grades with a table of IDs and keys. If yours does not match, the answer is wrong.

Brands is next, 10 points. The list returns every field of the table, with success and a **timestamp**. Get one by id gives 200, or 404 if it is not found.

Products is the big one, 65 points: list with page and **limit**, one product with a join, create, update and delete. The Exam Walkthrough builds all of it.

04

Run the Checker

Start the **server** on **port** 3000. Open a second terminal. Run the checker with your own ID.

terminal 2

```
$ node tests/test-runner.mjs 67000000000
```

The lecturer ran it on a project that did nothing. Every test failed, and that is fine. It shows you the questions.

checker output

```
  • [1. Identity] Student Identity Endpoint ... FAILED (0/10 pts)
    ↳ Reason: Expected HTTP 200, got 404
  • [2. Brands API] GET All Brands (List) ... FAILED (0/5 pts)
# ...one line per test, 12 tests
TOTAL SCORE              : 0 / 100 points
```

- Each failed test prints a Reason. It says what the checker expected and what it got, such as "expected 200, got 404". Read it, fix one thing, run again.
- Each test scores on its own. Do part of the spec and you still get part of the points.
- The checker is the same file for everyone. Your ID tells it which routes to call.

Here is a later run from the sample report: two tests fail, and the Reason points at the fix.

checker output, later run

```
  • [1. Identity] Student Identity Endpoint ... FAILED (0/10 pts)
    ↳ Reason: studentId mismatch. Expected '67000000000', got '12367000000000'
  • [3. Products Query] Summary DTO Filtering (GET /products) ... FAILED (0/10 pts)
    ↳ Reason: Summary DTO missing essential fields: 'id', 'sku', 'name'
TOTAL SCORE              : 80 / 100 points
```

The 100 points split like this.

- Identity 10. Brands 5 + 5.
- Products **Query** 35: **pagination** envelope 15, **summary DTO** 10, detail 10.
- Products Modify 30: create 10, update 10, delete 10.
- Standards 15: duplicate SKU 409, **JSON** envelope, clean route paths, 5 each.

> **A question from the class**
>
> A student asked whether 50 out of 100 here becomes 10 course points. The answer is not clear on the recording. Ask in the Teams channel.

05

Question 1 Live

The lecturer did question 1 live. It needs no layers. It is one route in app.js.

1. Make a variable for your ID.
2. Write app.get with the path /api/${ID}/student-info.
3. Copy the **response** shape from the spec. Do not retype it. Paste it, then put your own ID, name and key inside.
4. He put a fixed timestamp first. A real one comes from new Date().toISOString().
5. Restart the server. Run the checker again. Question 1 passes.

src/app.js

```
// 67000000000 - Somchai Jaidee
import express from 'express'
const STUDENT_ID = '67000000000'
const app = express()
// the spec says: every route starts with /api/<your id>
app.get(`/api/${STUDENT_ID}/student-info`, (req, res) => {
    res.status(200).json({
        success: true,
        data: { studentId: STUDENT_ID, name: 'Somchai Jaidee', secretKey: 'AB12cd' },
        timestamp: new Date().toISOString(),
    })
})
app.listen(3000, () => console.log('listening on port 3000'))
```

- He said it plainly: the real exam will ask for layers (**service**, **repository**, **mapper**). This question does not, so it is quick points.
- After the first run, restart the server every time you save, unless you use **node** --watch.

Question 2, brands, he left to the class: return everything, 200 for found, 404 for missing. Try it, then run the checker.

06

From Sample to Exam

The lecturer said the exam template looks like the last class: routes, services, repositories, a mapper. The class 8 sample is that project, built for classicmodels. For the exam you change names, not structure.

Database

classicmodels → mobile_distributor
db-config.js

Key column

productCode → id
every SQL with WHERE

Mapper fields

productName, MSRP, scale → sku, name, brandId, category, price, status
product-mapper.js

Duplicate check

productLine → sku
create() in the repository

Routes

offices, productlines, products → brands, products
app.js

The sample also has slips. Fix these before you trust it.

Offset

Uses pageSize, which is not capped. A limit above 100 skips the wrong rows.

Typo in office-route.js

It calls apiRespone.success. That name does not exist. Use apiResponse(res, data).

POST and PUT

They send bare JSON with no success or timestamp. The checker scores the envelope.

Password

db-config.js holds a password that is not yours. Put yours.

errorHandler

It is registered after listen. It works, but put it after the routes and before listen.

pagination-middleware.js

```
// pagination-middleware.js in the class 8 sample
const limit = Math.min(parseInt(pageSize), 100)
const offset = (page - 1) * pageSize   // wrong: pageSize is not capped
// fix: use the capped limit
const offset = (page - 1) * limit
```

product-route.js

```
// product-route.js: POST and PUT in the sample send bare JSON.
// The checker wants the same envelope as every other route.
import httpStatus from 'http-status'
router.post("/", async (req, res) => {
    const result = await service.createObject(req.body)
    await apiResponse(res, result, null, httpStatus.CREATED)
})
router.put("/:id", async (req, res) => {
    const result = await service.updateObject(req.params.id, req.body)
    await apiResponse(res, result)
})
```

product-repository.js

```
// product-repository.js: duplicate check on sku, not on productLine
export async function create(data) {
    const [dup] = await db.query('SELECT id FROM products WHERE sku = ?', [data.sku])
    if (dup.length > 0) {
        throw new AppError(`SKU ${data.sku} already exists`, httpStatus.CONFLICT)
    }
    const [result] = await db.query('INSERT INTO products SET ?', [data])
    return findOne(result.insertId)
}
```

- The POST **body** uses **snake_case** names such as brand_id and ram_gb. If they match the column names, INSERT … SET ? takes the body as it is.
- **Express** 5 sends a thrown error in an **async** route to the error handler. You do not need **try and catch** in each route.

> ⚠ **Check your columns**
>
> The table and column names in these snippets come from the spec fields. Compare them with your real schema, in your own database, before you copy.

07

Rules and Rehearsal

The lecturer's rules for the day, and what he asked you to do next.

- Offline. Run **npm** install once while you have a network. Then switch the network off. Turning it on again counts as cheating.
- Last year some students could not take the exam at all. That is why he ran this practice.
- Questions about the exam go in the Teams channel for the course, so everyone sees the answer. He will not answer private messages on Line.
- There is no class next week. You have two weeks to rehearse.

Rehearse in this order. Time yourself.

1. Unzip the template. Check the folder is not nested twice.
2. npm install, then switch the network off.
3. Load init-database.sql. Refresh. See two tables.
4. Set db-config.js and your ID. Make sure the server runs on port 3000.
5. Run the checker once to see the questions. Then do question 1.
6. Brands, then products. Run the checker after each endpoint.
7. Zip the work folder without node_modules. Name it your-id_exam.zip.

> **Do the dry run**
> In class nothing was submitted. On the day it will be. Practise the zip step too.

08

Glossary

async
: Start a slow job, keep working, and get called back when it finishes, instead of waiting.

body
: The data a request carries after the headers. For an API it is usually JSON.

checker

The script tests/test-runner.mjs. It calls your running server, checks each endpoint against the spec and prints a score.

connection

An open line between your program and the database. You need one before you can send any SQL.

DTO

A shaped copy of an entity for the API: only the fields the client needs, in the format you agreed.

endpoint

One method plus path an API answers, such as GET /api/subjects.

Express

A small web framework for Node.js. It adds routing and middleware on top of the built-in http module.

JSON

JavaScript Object Notation: a plain text format for data that both humans and programs can read.

limit

How many records one page holds. The client sends it as limit, and the server caps it.

mapper

A class that turns an entity into a DTO, so the conversion lives in one reusable place.

node

A runtime that runs JavaScript outside the browser, so the same language can write the back end.

npm

Node Package Manager: installs libraries such as express into node_modules.

pagination

Returning a big list one page at a time, so responses stay small and fast.

PATH

A Windows setting that lists the folders searched when you type a command. If the MySQL bin folder is not in it, mysql is not recognized.

port

A number that says which program on the machine should get the message. Web servers use 80 by default.

Query

The part of the URL after ?, like page=2&size=10. Used to filter or page a list.

repository

The layer that reads and writes the database. No business rules, no HTTP.

resource

A thing the API gives access to, such as a product or a customer. In this course it maps to a database table.

response

What the server sends back: status code, headers and a body.

route

Matching an HTTP method and a URL path to the function that handles it.

server

A program (or the machine running it) that waits for requests and answers them.

service

The layer that holds the business rules. It calls repositories and knows nothing about HTTP.

snake_case

Words joined by underscores, like user_name. Common for database columns, not for URIs.

summary

Two sizes of DTO: summary has a few fields for lists, detail has everything for one item.

timestamp

A date and time value. A column like last_update can fill itself with the current time.

try and catch

try runs code that may fail. If it throws, execution jumps to catch, which receives the error.